

ASSIGNMENT 1

1. Describe your project.

The AI Chatbot for College is a web-based application that instantly answers common student questions such as admission details, fee structure, exam timetable, attendance rules, library timings, course information, faculty contact details, branch/course change procedures, placements, hostel accommodation, scholarships, and campus facilities. It uses rule-based keyword matching with typo-tolerant fuzzy matching to understand what a student is asking, then fetches the correct answer from a database. Students can log in with their registration number to get personalized answers about their own attendance, fees, CGPA, assigned mentor, or exam date. The system also includes voice typing and read-aloud features, dark mode, an installable Progressive Web App configuration, and a secure admin command center with analytics and CSV export.

2. Advantages of your project.

- Provides instant, 24x7 answers without needing to visit the college office.
- Reduces repetitive workload on administrative staff.
- Tolerates minor spelling mistakes through fuzzy matching.
- Personalized answers for logged-in students (attendance, fees, CGPA, mentor, exam date).
- Voice-enabled with Speech-to-Text and SpeechSynthesis read-aloud support.
- Admin can update FAQs anytime through a secure dashboard, with no coding needed.
- Chat history is logged with student feedback ratings (👍/👎), helping identify areas for improvement.
- Installable as an app on phone or laptop (Progressive Web App), working like a native application without needing an app store.

3. How do you gather the requirements of your project?

Requirements were gathered by identifying the most common questions students ask the college office repeatedly — admission process, fee structure, exam schedules, attendance policy, library timings, course details, placement records, hostel facilities, and faculty contacts. These were confirmed as realistic, high-frequency queries based on general college administrative experience, and then structured into an indexed knowledge base of 14 categories. Additional requirements (student login for personalized data, admin panel with analytics, CSV export, voice accessibility, secure authentication) were identified as necessary to make the system practically usable and maintainable by non-technical college staff.

4. How and where do you implement your project?

The project is implemented using Python (Flask) for the backend, HTML/CSS/JavaScript for the frontend, and SQLite for the database. It was developed locally and then version controlled using Git and hosted on GitHub. From GitHub, it is deployed to Render, a cloud hosting platform, which automatically rebuilds and redeploys the application every time new code is pushed to the repository. This gives the project a live, publicly accessible URL (<https://college-ai-chatbot-c94w.onrender.com>) that works from any device with an internet connection.

5. Which software you used in your project?

- VS Code — code editor used to write and edit the project files.
- Python 3 — programming language for the backend logic.
- Flask (v3.0.3) — Python web framework used to build the server and routes.
- SQLite3 — lightweight relational database used to store FAQs, students, admin, and chat history.
- Werkzeug (v3.0.3) — security library for cryptographic password hashing (PBKDF2/scrypt).
- Git and GitHub Desktop — version control and pushing code to GitHub.
- Render — cloud platform used to host and deploy the live application.
- Google Chrome / Safari — used for testing the chatbot interface and Web Speech APIs.

6. What methodology is used for the project?

The project follows an incremental/iterative development methodology. The system was built in stages: first a working core chatbot with keyword matching, then a database schema, then student login, then the admin panel, then security improvements (password hashing), then UI polish and fuzzy matching, then PWA support, and finally voice input/output, dark mode, expanded student records (CGPA/mentor), and CSV analytics export. Each stage was tested and confirmed working before adding the next feature, which is consistent with an iterative/incremental approach.

7. What is the hardware or software of the project?

Hardware requirements are minimal: any computer or smartphone with a web browser and an internet connection can access the chatbot, since all processing happens on the server side (Render's cloud servers), not on the user's device. Software requirements include: Python 3 and Flask (backend framework), SQLite (database engine), a modern web browser (Chrome, Safari, Edge, etc.) supporting HTML5/CSS3/JavaScript and the Web Speech API, and Werkzeug for secure password hashing. No special hardware, GPU, or paid software licenses are required.

8. What is your role in the project?

As part of a two-member team, my role covered core backend development (Flask routes, chatbot matching logic, database schema and queries), database design (defining the college_info, students, chat_history, and admin tables), implementing student and admin authentication with secure password handling, voice input/output integration, automated unit testing, and deploying the project using Git, GitHub, and Render. I was also responsible for testing each feature end-to-end before moving to the next stage of development.

9. Explain the data flow of your application.

When a student types a message, speaks via voice input, or taps a quick-reply button, the browser sends it to the Flask backend via the /chat endpoint as JSON. The backend first checks whether the student is logged in and asking about their own data (e.g. "my attendance", "my cgpa"); if so, it queries the students table directly. Otherwise, it runs the message through the match_query() function, which checks it against the keywords column of every row in the college_info table, using exact matching first and fuzzy matching as a fallback. The best matching answer is retrieved, the query-response pair along with row ID is saved into the chat_history table, and the response is sent back to the browser as JSON, where JavaScript renders it in the chat window (including converting any table-formatted answers into an actual HTML table and triggering audio chimes).

10. What are the entities in your project?

- **College Info** : id (PK), category, keywords, and answer for each FAQ topic.
- **Student** : roll_no (PK), name, attendance percentage, fees due, exam date, course, semester, cgpa, and mentor.
- **Chat History** : id (PK), roll_no (FK), user query, bot response, timestamp, and feedback rating.
- **Admin** : username (PK) and securely hashed password for the admin panel.

11. What is the drawback of your system?

The chatbot relies on rule-based keyword and fuzzy matching rather than open-domain generative AI, so it can occasionally misinterpret questions that are phrased in an unusual way. It depends on the admin keeping the FAQ database accurate and up to date, since it cannot generate new information on its own. The system also requires an internet connection to function, and the free hosting tier can take up to 30–40 seconds to “wake up” after a period of inactivity.

12. How do you test your application?

The application was tested through both automated and manual methods. An automated unit test suite (test_suite.py) was implemented using Python's unittest framework, verifying query matching, typo-tolerant fuzzy thresholds (e.g. “hostal” → Hostel), and personalized student queries (attendance, CGPA, mentor) with a 100% pass rate. Manual testing was performed across desktop and mobile browsers (Chrome and Safari) to verify voice recognition, speech synthesis, dark mode toggling, student login validation, admin CRUD operations, and CSV log exports. The deployed live version on Render was also tested directly after each Git commit.

13. From where do you get reference, guide for your project work?

Reference and guidance were drawn from the official Flask and SQLite documentation for backend and database concepts, MDN Web Docs for HTML/CSS/JavaScript and Web Speech API specifications, and Render's deployment documentation for setting up continuous deployment from GitHub. The overall project structure, database data, and problem statement were guided by common, real-world administrative pain points observed in college settings.

14. In the future, what changes will you make in your application?

- Connect to Generative AI / LLMs (Google Gemini or OpenAI API) with Retrieval-Augmented Generation (RAG) for conversational depth.
- Add official WhatsApp Business integration for access without opening a browser.
- Add multi-language conversational support in Hindi and Marathi alongside English.
- Real-time automated synchronization with the college ERP database to update attendance and fee clearances automatically.